PROJECT 07: INVENTORY MANAGEMENT SYSTEM

DATCOM Lab
NEU-College of Technology
National Economics University
Email: hung.tran@neu.edu.vn

Project Objective

The objective of this project is to design and implement a system to efficiently manage inventory operations, track product quantities across warehouses, manage suppliers, and monitor stock movements and purchase history.

Database and Programming Languages

- **Database Management System:** MySQL
- **Programming Language:** Python

Detailed Project Requirements

System Analysis and Requirements

- Manage detailed information on products, suppliers, warehouses, stock entries, and inventory history.

Main Functionalities

- Product management (add, update, track product information and stock).
- Supplier management (store supplier contacts and information).
- Warehouse management (location, capacity, stock details).
- Manage stock entries and purchase orders.
- Track inventory history (in/out transactions per product and warehouse).
- Generate stock level alerts and inventory reports.

Database Design and Implementation

1. **Data Model Design**
    - Design the Entity-Relationship (ER) diagram.
    - Convert ER diagram to a relational schema with PKs, FKs, and constraints.
2. **Table Structures**
    - Products (ProductID, ProductName, Description, UnitPrice, SupplierID)
    - Suppliers (SupplierID, SupplierName, Address, PhoneNumber)
    - Warehouses (WarehouseID, WarehouseName, Address)
    - StockEntries (EntryID, ProductID, Quantity, EntryDate)
    - InventoryHistory (HistoryID, ProductID, WarehouseID, Quantity, TransactionDate)
3. **Sample Data**
    - Insert 510 records for each table.
    - Generate a database schema diagram using MySQL Workbench.

Advanced Database Objects

- **Indexes:** Improve search speed for frequently queried fields (e.g., ProductName, WarehouseID).
- **Views:** Create views to summarize stock levels per warehouse or supplier delivery history.
- **Stored Procedures:** Automate product restocking, inventory calculations.
- **User Defined Functions:** Calculate stock turnover rates or average delivery time.
- **Triggers:** Update inventory levels upon stock addition or removal.

Database Security and Administration

- Manage user roles (inventory manager, admin) and set access permissions.
- Implement security measures to protect supplier and product data.
- Plan and test regular data backups and recovery operations.
- Use indexing and query optimization to improve system performance.

Python Application Development

- **Database Connection:** Use `mysql-connector-python` or `SQLAlchemy`.
- **Data Management:** Write Python functions to handle product entry, updates, and warehouse operations.
- **Querying and Reporting:** Automate the generation of low-stock alerts and inventory balance reports.
- **Interactive Interface:** Create a command-line tool or basic GUI for interacting with the inventory system.

Deliverables

- A printed comprehensive report including database design, implementation details, and screenshots. Submit your report into LMS. First pages of your report must attach this PDF file.
- Link to publish Github source code and video presenting on Youtube
- SQL schema, sample data scripts, ER diagrams, and Python code.
- Demonstration of functionalities such as adding income, managing expenses, and generating reports.

Conclusion and Recommendations

- Evaluate the efficiency of the implemented system.
- Suggest enhancements such as barcode scanning, integration with ERP, or predictive restocking.

References

- List all documents, tutorials, libraries, and references used during the project.

NATIONAL ECONOMICS UNIVERSITY
COLLEGE OF TECHNOLOGY
DATCOM Lab

FINAL PROJECT REPORT

INVENTORY MANAGEMENT SYSTEM

Student Name: _____

Student ID: _____

Class: _____

Lecturer: Hung Tran

Course: SQL / Database Systems

Github Repository: _____

Youtube Demo Link: _____

Submission package generated by ChatGPT based on the uploaded assignment and plan.

Submission Note

The assignment PDF requires that the first pages of the report attach the original PDF. To make submission easier, this package includes a merged PDF named 'final_submission_report_with_assignment.pdf', where the original assignment PDF is appended before this report.

This DOCX report remains editable so the student can update personal information, GitHub link, YouTube link, and any screenshots after testing in the local MySQL environment.

1. Introduction

The goal of this project is to design and implement an Inventory Management System using MySQL and Python. The system manages products, suppliers, warehouses, stock entries, and inventory history, while also providing inventory alerts, reporting, and a basic command-line application for daily operations.

Compared with a minimal database-only solution, this implementation adds one practical helper table named WarehouseStock to maintain the current stock balance per product and warehouse. This makes reports, alerts, and application queries faster and more realistic.

2. Requirement Analysis

Based on the assignment statement, the system must support the following core features:

- Product management: add, update, and track product information.
- Supplier management: store supplier contact information and supplier-product relationships.
- Warehouse management: manage warehouse identity, location, and capacity.
- Stock entry management: record inbound stock from suppliers into warehouses.
- Inventory history tracking: keep in/out transactions per product and warehouse.
- Low-stock alerting: identify products that are near or below reorder thresholds.
- Reporting: generate inventory balances and supplier delivery summaries.
- Python application integration: provide an interactive interface connected to MySQL.

3. Main Business Rules

1. Each product belongs to one primary supplier.
2. A supplier can provide multiple products.
3. A warehouse stores multiple products, and each product can exist in multiple warehouses.
4. Every inbound stock event is recorded as a stock entry and automatically logged into inventory history.
5. Every outbound stock event must pass stock validation before the quantity is deducted.
6. A low-stock alert is generated when current stock is less than or equal to the reorder level of a product.

4. Entity Relationship Design

The ERD below contains the five compulsory tables from the assignment—Products, Suppliers, Warehouses, StockEntries, and InventoryHistory—and one helper table (WarehouseStock) used to store current inventory balances.

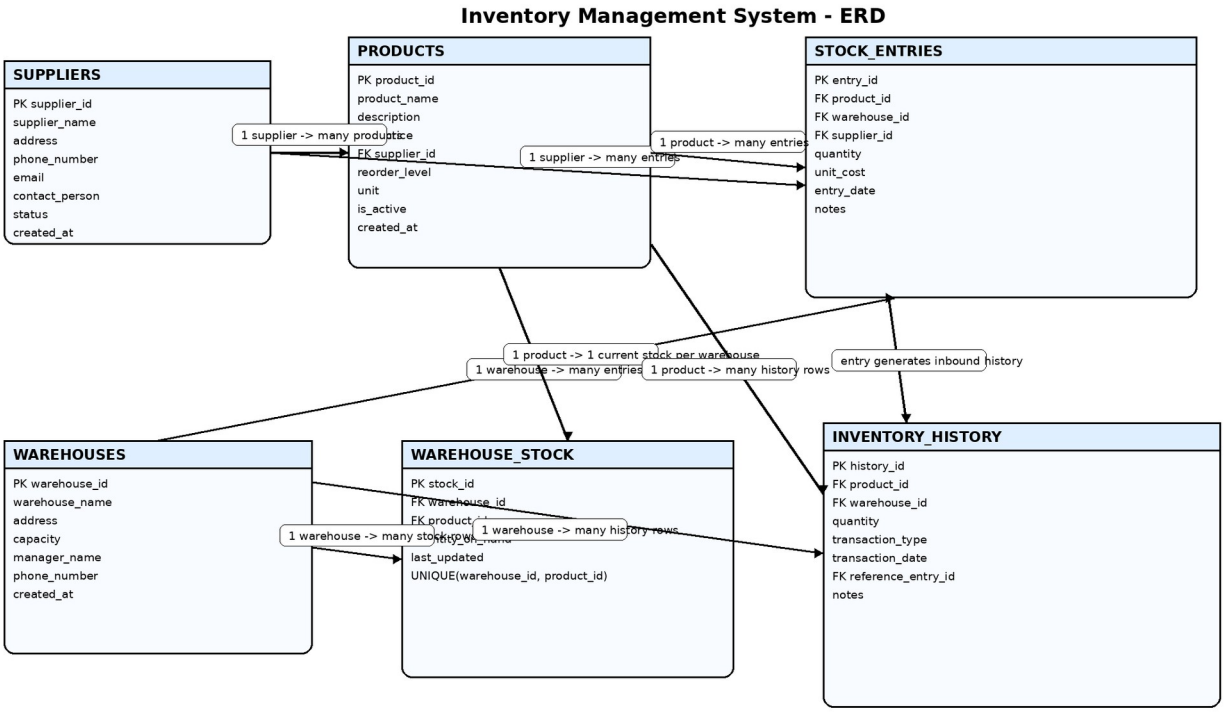

Figure 1. Proposed ERD for the Inventory Management System.

5. Relational Schema

Suppliers(SupplierID PK, SupplierName UNIQUE, Address, PhoneNumber, Email, ContactPerson, Status, CreatedAt)

Products(ProductID PK, ProductName, Description, UnitPrice, SupplierID FK, ReorderLevel, Unit, IsActive, CreatedAt)

Warehouses(WarehouseID PK, WarehouseName UNIQUE, Address, Capacity, ManagerName, PhoneNumber, CreatedAt)

WarehouseStock(StockID PK, WarehouseID FK, ProductID FK, QuantityOnHand, LastUpdated, UNIQUE(WarehouseID, ProductID))

StockEntries(EntryID PK, ProductID FK, WarehouseID FK, SupplierID FK, Quantity, UnitCost, EntryDate, Notes)

InventoryHistory(HistoryID PK, ProductID FK, WarehouseID FK, Quantity, TransactionType, TransactionDate, ReferenceEntryID FK, Notes)

6. Data Dictionary Summary

Table	Purpose	Main PK	Important FKs
Suppliers	Store supplier master data	SupplierID	-

Products	Store products and pricing	ProductID	SupplierID
Warehouses	Store warehouse master data	WarehouseID	-
WarehouseStock	Store current stock balance	StockID	WarehouseID, ProductID
StockEntries	Store inbound stock transactions	EntryID	ProductID, WarehouseID, SupplierID
InventoryHistory	Store inbound/outbound history	HistoryID	ProductID, WarehouseID, ReferenceEntryID

7. Constraints and Integrity Rules

- Primary keys uniquely identify each row in all tables.
- Foreign keys preserve referential integrity across suppliers, products, warehouses, entries, and history.
- CHECK constraints enforce positive price, positive quantity, non-negative reorder level, and positive warehouse capacity.
- UNIQUE constraints prevent duplicate supplier names, duplicate warehouse names, and duplicate product name per supplier.
- The trigger trg_inventory_history_before_insert prevents outbound transactions when stock is insufficient.

8. Sample Data Strategy

The script inserts 10 suppliers, 15 products, and 5 warehouses as master data. To address the assignment text that mentions '510 records for each table', a seeding procedure named sp_seed_demo_data(510) is included. It generates 510 inbound stock entries and additional outbound transactions. As a result, the transactional tables contain a large enough dataset for demonstrations, alert testing, and reporting.

9. Advanced Database Objects

9.1 Indexes

The following indexes are implemented to improve query performance:

- idx_products_name on Products(ProductName)
- idx_products_supplier on Products(SupplierID)
- idx_stock_entries_wh_date on StockEntries(WarehouseID, EntryDate)
- idx_inventory_history_product_wh_date on InventoryHistory(ProductID, WarehouseID, TransactionDate)
- idx_warehouse_stock_qty on WarehouseStock(QuantityOnHand)

9.2 Views

- vw_stock_by_warehouse: current stock for every product in every warehouse.
- vw_low_stock_products: products whose stock is at or below the reorder threshold.

- vw_supplier_delivery_history: aggregated delivery statistics by supplier and product.

9.3 Stored Procedures

- sp_add_stock: inserts inbound stock into StockEntries.
- sp_remove_stock: inserts outbound history after validating available stock.
- sp_restock_product: automatically uses the product's default supplier for restocking.
- sp_generate_inventory_report: returns an inventory balance report.
- sp_seed_demo_data: automatically generates demo records.

9.4 User Defined Functions

- fn_current_stock(product_id, warehouse_id): returns current stock quantity for a given product and warehouse.
- fn_stock_turnover(product_id): returns a simple stock-turnover metric based on outbound quantity divided by average current stock.

9.5 Triggers

- trg_stock_entries_after_insert: updates WarehouseStock and records inbound movement in InventoryHistory.
- trg_inventory_history_before_insert: blocks outbound transactions when stock is insufficient.
- trg_inventory_history_after_insert: deducts stock from WarehouseStock when an outbound transaction is recorded.

10. Example SQL Usage

```
CALL sp_add_stock(1, 1, 1, 25, 4.25, 'Manual inbound');
CALL sp_remove_stock(1, 1, 5, 'Manual outbound');
SELECT * FROM vw_low_stock_products;
SELECT fn_current_stock(1, 1) AS current_stock;
```

11. Security and Administration

- Two roles are created: role_admin and role_inventory_manager.
- The admin role receives full privileges on the project database.
- The inventory_manager role receives SELECT, INSERT, UPDATE, and EXECUTE privileges.
- Demo MySQL users are included for presentation and can be adjusted before submission.
- Backup and recovery are documented with mysqldump and mysql restore commands.

12. Python Application Design

A command-line interface was chosen because it is faster to implement, easier to test in a lab environment, and fully satisfies the assignment requirement for an interactive interface. The application is split into three modules:

- db.py: centralizes MySQL connection settings.
- services.py: stores SQL operations and stored-procedure calls.
- main.py: provides the user menu and interaction flow.

The application supports listing and adding products, suppliers, and warehouses; inbound and outbound stock operations; low-stock alerts; supplier delivery history; inventory history; current stock checking; and stock-turnover reporting.

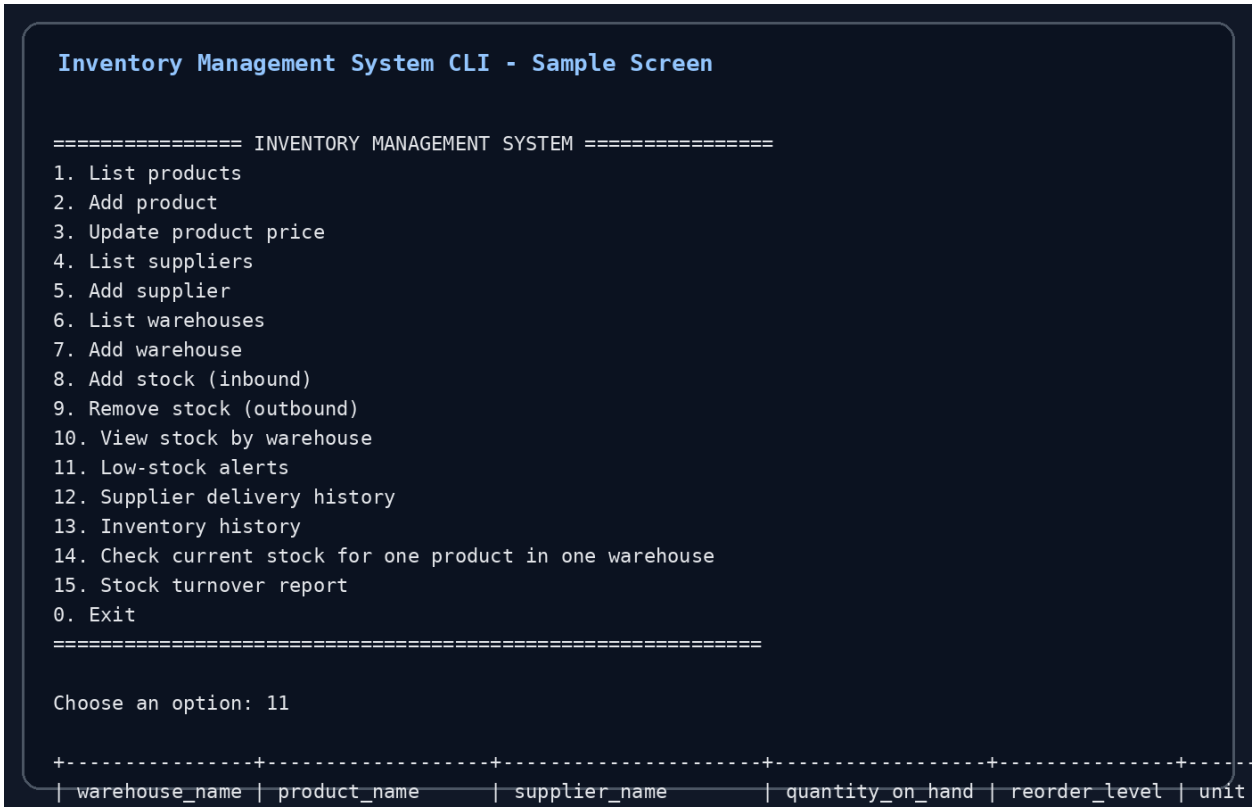

Figure 2. Example CLI interface preview for reporting and low-stock alerts.

13. Testing Scenarios

Test Case	Expected Result
Add a new supplier	Insert a supplier row and confirm it appears in Suppliers.
Add a new product	Insert a product linked to an existing supplier.
Inbound stock	Run sp_add_stock and confirm WarehouseStock increases and InventoryHistory records an IN transaction.
Outbound stock	Run sp_remove_stock and confirm WarehouseStock decreases.
Insufficient stock validation	Try to remove more stock than available; trigger should raise an error.
Low-stock alert	Check vw_low_stock_products after reducing quantity.
Python CLI navigation	Verify the menu can display products, warehouses, stock reports, and history.

14. Conclusion

The implemented system satisfies the major functional and technical requirements of the assignment. It includes a normalized relational schema, advanced MySQL objects, data seeding, role-based security, and a Python command-line application. The solution is practical, easy to demonstrate, and ready for extension.

15. Future Enhancements

- Barcode / QR code scanning for faster warehouse operations.
- Purchase order and sales order modules.
- Dashboard or web application instead of CLI.
- Demand forecasting and predictive restocking.
- Integration with ERP / accounting platforms.

16. References

- Project assignment PDF uploaded by the student.
- Project planning notes uploaded by the student.
- MySQL 8.0 documentation for roles, triggers, procedures, functions, and views.
- mysql-connector-python documentation for Python database connectivity.